EC413 Lab 7: Pipelined CPUs

Jatin Punjabi and Jeehan Zaman

November 13, 2026

1 Overview

The supplied design is a standard five-stage MIPS pipeline (IF, ID, EX, MEM, WB) with no forwarding, so an instruction that reads a register written by one of the previous two instructions reads a stale value. This lab adds "1 ahead" and "2 ahead" forwarding, arbitration between them, protection for writes to `$0` and for instructions that do not write a register, a register-file bypass, and (extra credit) forwarding of store data for a `lw` followed by `sw`.

All changes were simulated with Icarus Verilog using the lab's provided testbench and a self-checking testbench of my own.

2 Tasks 2–4: Forwarding unit (1 ahead, 2 ahead, arbitration)

A new module, `forwardingunit`, sits in the EX stage. It compares the source registers held in ID/EX (`Rs`, `Rt`) with the destination registers in EX/MEM and MEM/WB and drives `ForwardA` and `ForwardB`. In the top-level `CPU` module two 3-to-1 muxes (`ForwardAMux`, `ForwardBMux`) were inserted between the ID/EX register and the ALU. The existing wires `MuxASelect` and `MuxBSelect` carry the selects.

Select	Source	Meaning
`00`	ID/EX register value	no hazard
`01`	`WriteBackData` (after the MemToReg mux)	2 ahead
`10`	`MemAluOut` (EX/MEM ALU result)	1 ahead

Table 1: Forwarding mux encoding.

Task 2 (1 ahead). If EX/MEM writes a register and `EXMEM_Rd` equals `IDEX_Rs` (or `Rt`), select `10`.

Task 3 (2 ahead). If MEM/WB writes a register and `MEMWB_Rd` equals `IDEX_Rs` (or `Rt`), select `01`. The write-back *mux output* is used, so values that come from a load are also forwarded correctly.

Task 4 (arbitration). When both stages match, the instruction in EX/MEM is the more recent writer, so 1 ahead takes priority; 2 ahead is chosen only if 1 ahead does not match.

Datapath changes. The ALU's A input is driven by `ForwardAMux`; input 0 of the existing AluSrc mux (previously ID/EX `B`) is driven by `ForwardBMux`, so immediates still override `B`. The EX/MEM store-data register is loaded from the forwarded `B` value, so a `sw` also receives forwarded data.

3 Task 5: Writes to $0

Register `$0` is hard-wired to zero, but `add $0,$R10,$R10` still has `Rd=0` and `RegWrite=1`, and would be forwarded. Every forwarding condition therefore also requires `Rd != 0`. *Test:* `add $0,$R10,$R10` followed by `add $R11,$0,$R10` yields R11 = 5 (it would be 15 if the `$0` result were forwarded).

4 Task 6: No write

Instructions such as `sw`, `beq`, `bne`, `j` and pipeline bubbles have `RegWrite=0`, but the register field in their encoding can still match a later source register (and a `sw`'s ALU output is only an address). Every forwarding condition therefore requires the producer's RegWrite bit (`EXMEM_WBControl[0]` for 1 ahead, `WB_WBControl[0]` for 2 ahead). *Test:* `sw $R2,8($R0)` followed by `add $R12,$R2,$R2` yields 184 (it would be 16 if the `sw`'s address were forwarded).

5 Task 7: Register-file bypass

A value produced three instructions earlier is being written in WB in the same cycle the consumer reads the register file in ID, so forwarding cannot help; the register file must pass the write data straight through. The supplied register file had write-through, but it ignored `RegWrite` and `RegWrite` was missing from the sensitivity list, so a non-writing instruction could wrongly bypass. It now bypasses only when `RegWrite && WriteRegister == ReadReg && ReadReg != 0`.

6 Extra credit: LW followed by SW

A `sw` immediately after the instruction that produces its store data reaches MEM while the producer is in WB, so the data is only available as `WriteBackData`. No stall is needed:

- the `Rt` field is carried one more stage (new register `EXMEM_RtReg`);
- new module `storeforward` asserts `ForwardStore` when `MemWrite && RegWrite_WB && WB_Rd != 0 && WB_Rd == EXMEM_Rt`;
- a new mux, `StoreDataMux`, in front of the data-memory write-data port selects `WriteBackData` instead of the EX/MEM store-data register.

Test: `lw $R17,8($R0); sw $R17,20($R0)` stores 92 into Mem[20]. A `lw`, `nop`, `sw` sequence is covered by the ordinary 2-ahead forward on B and also stores the correct value.

Limitation. A `lw` followed directly by a non-store instruction that uses the loaded value (a load-use hazard) still requires a one-cycle stall. The hazard detection unit is a stub in the supplied project, and this case is outside the lab.

7 Modified datapath

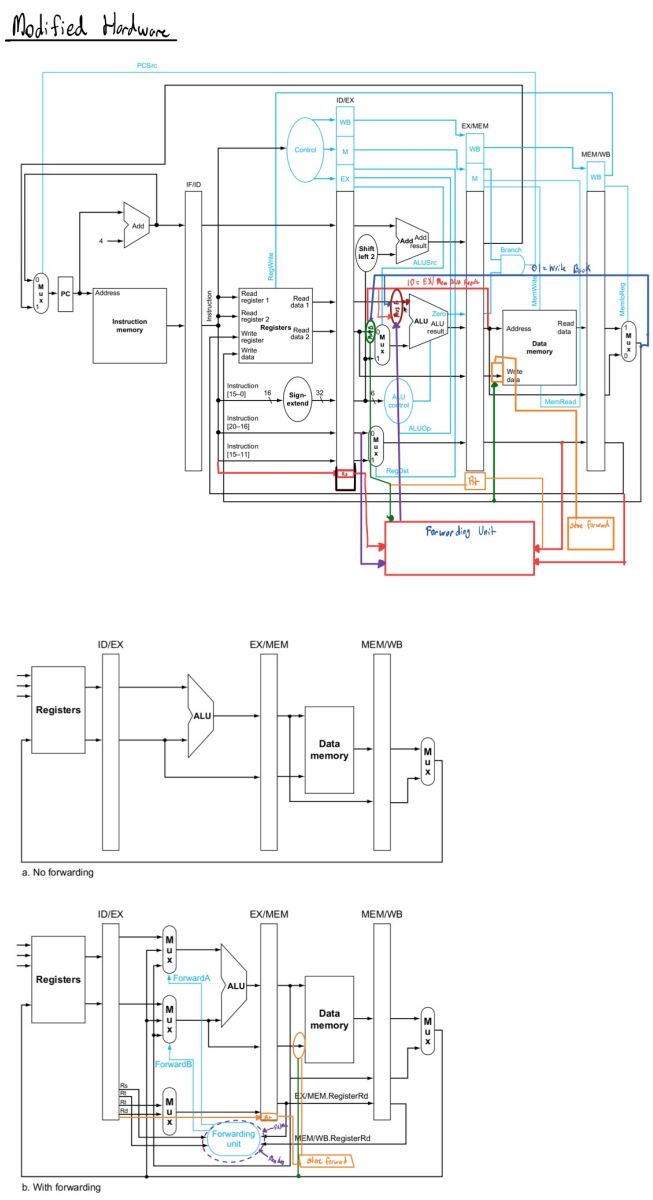


Figure 1: Modified datapath: forwarding muxes and unit, store-data forwarding, and register-file bypass.